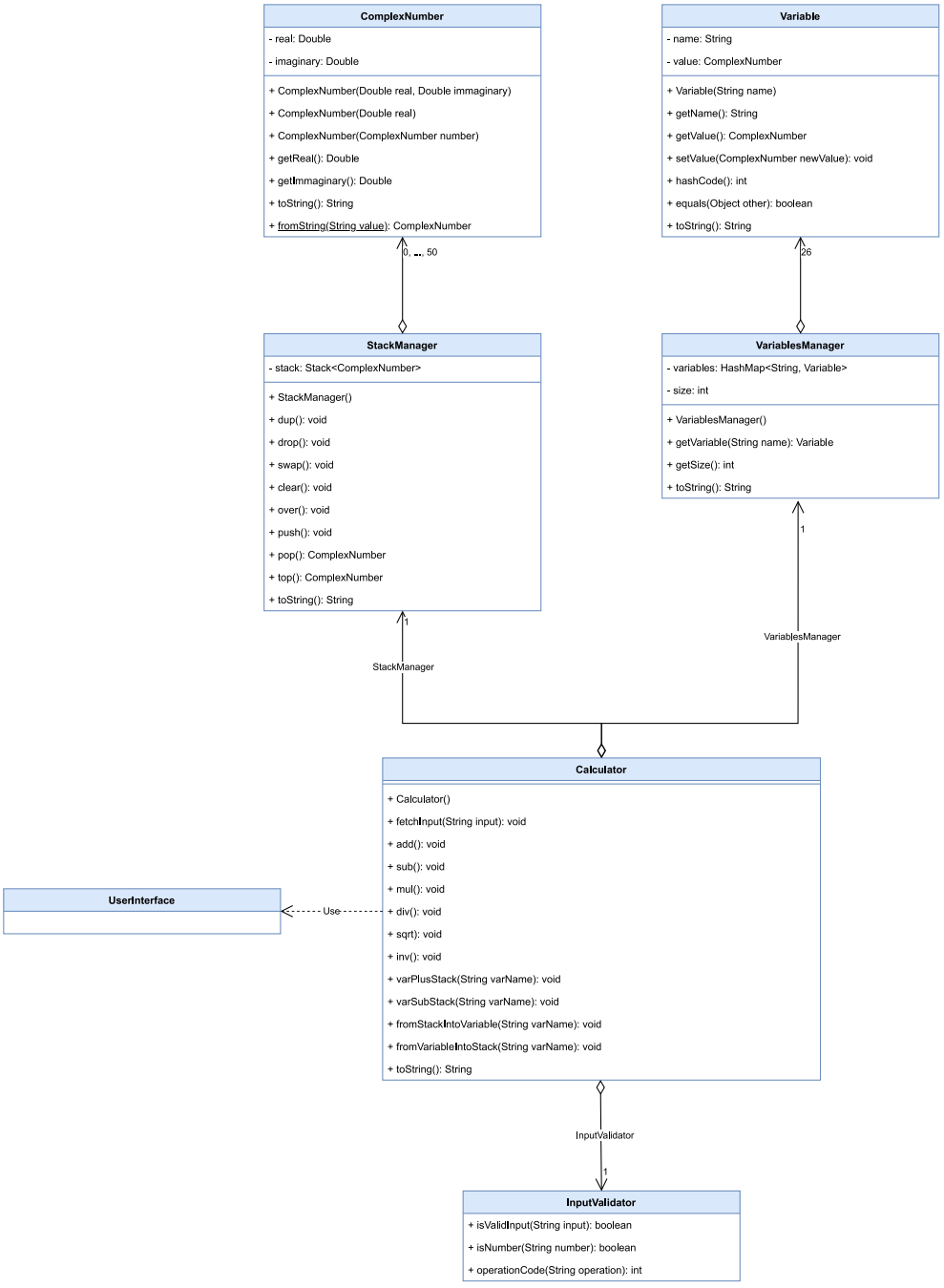
Design

Membri del gruppo (Gruppo-03):

- Francesco Lemmo - 0612705508 (Group leader)
- Leonardo Luongo - 0612706039
- Francesco Monda - 0612705277
- Mattia Vitali - 0612706221

Durante la fase di Requirements Engineering abbiamo affrontato il "*Cosa*", cioè cosa dovrebbe fare il sistema, nella fase di Design affrontiamo il "Come", ovvero come funziona il sistema e i singoli componenti, e come essi si interfacciano e interagiscono tra loro.

Class Diagram



Classes description

ComplexNumber

La classe descrive i numeri complessi attraverso un attributo per la parte reale (real) ed un attributo per la parte immaginaria (imaginary), entrambi privati.
La classe mette a disposizione i seguenti metodi:

- **`public ComplexNumber(Double real, Double imaginary)`** - Costruttore
  - **`real`** – parte reale
  - **`imaginary`** – parte immaginaria
- **`public ComplexNumber(Double real)`** – Costruttore sovraccarico per l'inserimento di numeri reali
  - **`real`** – parte reale
- **`public ComplexNumber(ComplexNumber number)`** – Costruttore sovraccarico per l'inserimento di numeri complessi
  - **`number`** – numero complesso
- **`public Double getReal()`** – Restituisce la parte reale
- **`public Double getImaginary()`** - Restituisce la parte immaginaria
- **`public String toString()`** – Rappresenta il numero complesso come una stringa
- **`public static ComplexNumber fromString(String value)`** – È un metodo statico che, prendendo una stringa come argomento la traduce in numero complesso, restituendolo
  - **`value`** – stringa da convertire

Variable

La classe rappresenta una singola variabile messa a disposizione dell'utente, essa è contraddistinta da un nome (name) e da un valore, un numero complesso, (value).
La classe mette a disposizione i seguenti metodi:

- **`public Variable(String name)`** - Costruttore
  - **`name`** – nome della variabile
- **`public String getName()`** – Restituisce il nome della variabile
- **`public ComplexNumber getValue()`** - Restituisce il valore della variabile
- **`public void setValue(ComplexNumber newValue)`** – Inserisce un nuovo valore all'interno della variabile
- **`public int hashCode()`** - Calcola l'hash code della variabile. L'hash code dipende solo dal nome della variabile
- **`public boolean equals(Object other)`** - Verifica se la variabile è uguale ad un altro oggetto. Restituisce true se l'altro oggetto non è null, se è della stessa classe dell'oggetto corrente e se hanno lo stesso nome (case insensitive).
  - **`other`** – l'altro oggetto
- **`public String toString()`** – Rappresenta la variabile come una stringa

StackManager

La classe gestisce la struttura dati dello stack che può contenere numeri complessi, per un massimo di 50 di essi.
La classe mette a disposizione i seguenti metodi:

- **`public StackManager()`** - Costruttore
- **`public void dup()`** – Inserisce una copia dell'ultimo elemento
- **`public void drop()`** – Elimina l'ultimo elemento
- **`public void swap()`** – Inverte gli ultimi due elementi
- **`public void clear()`** – Svuota lo stack
- **`public void over()`** – Inserisce una copia del penultimo elemento

- **`public void push()`** – Inserisce un elemento nello stack
- **`public ComplexNumber pop()`** – Estrae l'ultimo elemento dallo stack e lo restituisce
- **`public ComplexNumber top()`** – Restituisce una copia dell'ultimo elemento dello stack
- **`public String toString()`** – Rappresenta lo stack come una stringa

VariablesManager

La classe gestisce la struttura dati che contiene le 26 variabili (allocate, ma da inizializzare) ed i metodi di accesso ad essa.
La classe mette a disposizione i seguenti metodi:

- **`public VariablesManager()`** - Costruttore
- **`public Variable getVariable(String name)`** – Restituisce la variabile associata ad un determinato nome
  - **`name`** – nome della variabile da restituire
- **`public int getSize()`** – Restituisce il numero di variabili presenti
- **`public String toString()`** – Rappresenta l'insieme delle variabili come una stringa

InputValidator

La classe implementa il controllo dell'input e ne riconosce la validità e la tipologia.
La classe mette a disposizione i seguenti metodi:

- **`public boolean isValidInput(String input)`** – Verifica che l'input in arrivo sia valido
  - **`input`** – stringa in input
- **`public boolean isNumber(String number)`** – Verifica che l'input in arrivo sia un numero
  - **`number`** – numero da valutare in input
- **`public int operationCode(String operation)`** – Verifica che in input ci sia un'operazione e ne restituisce il codice numerico
  - **`operation`** – operazione da valutare in input

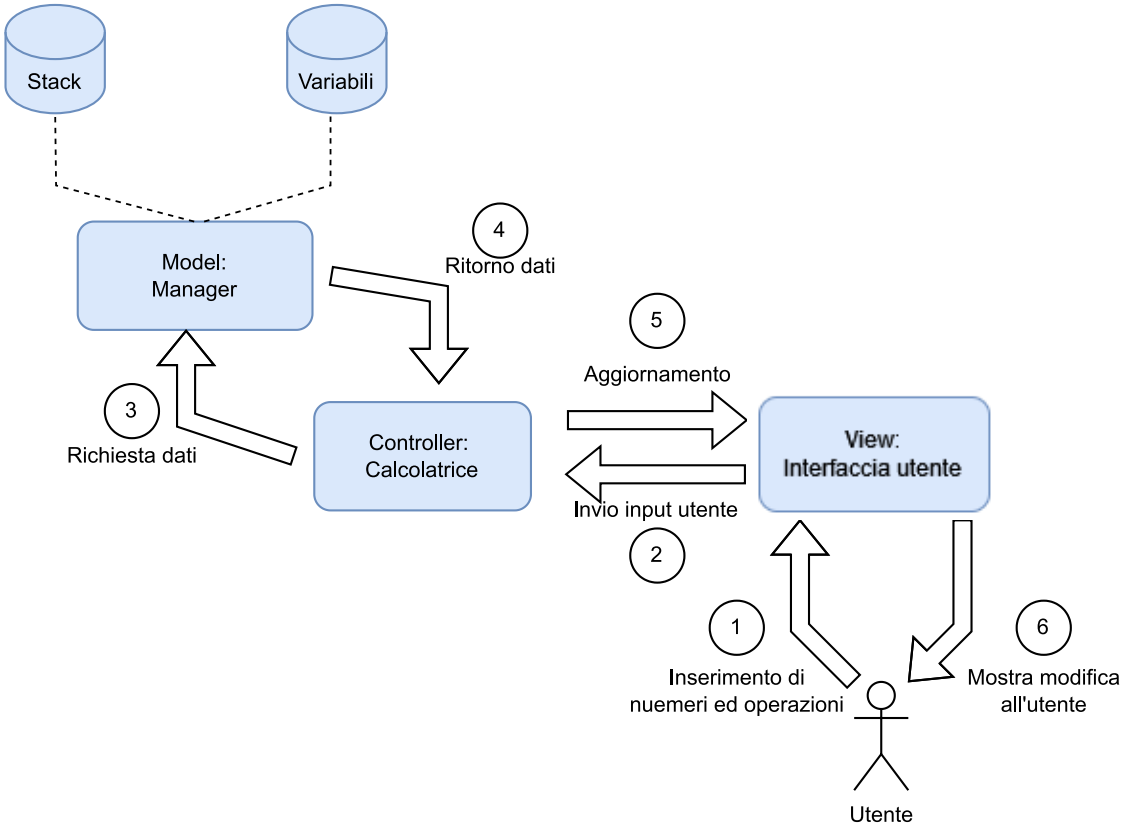
Calculator

La classe calculator compone il core del sistema e assume il ruolo principale come controller di esso. All'interno della classe è presente uno StackManager (permettendo di gestire le operazioni sullo stack), un VariablesManager (per l'utilizzo delle variabili) ed un InputValidator (al fine di controllare gli input).
La classe mette a disposizione i seguenti metodi:

- **`public Calculator()`** - Costruttore
- **`public boolean fetchInput(String input)`** – il metodo richiama l'InputValidator e, dopo aver riconosciuto il tipo di input, esegue l'inserimento o l'operazione desiderata. Il metodo fetchInput ed il suo relativo funzionamento sono legati all'utilizzo della calcolatrice da console, senza la presenza di alcuna interfaccia grafica.
- **`public void add()`** – Preleva gli ultimi due elementi dello stack, ne fa la somma ed inserisce il risultato nello stack.
- **`public void sub()`** – Preleva gli ultimi due elementi dello stack, ne fa la differenza ed inserisce il risultato nello stack.
- **`public void mul()`** – Preleva gli ultimi due elementi dello stack, ne fa la moltiplicazione ed inserisce il risultato nello stack. Restituisce il risultato
- **`public void div()`** – Preleva gli ultimi due elementi dello stack, ne fa la divisione ed inserisce il risultato nello stack.
- **`public void sqrt()`** – Preleva l'ultimo elemento dello stack, ne fa la radice quadrata ed inserisce il risultato nello stack.

- **`public void inv()`** – Preleva l'ultimo elemento dello stack, ne inverte il segno ed inserisce il risultato nello stack.
- **`public void varPlusStack(String varName)`** – Preleva l'ultimo elemento dello stack e lo somma al valore della variabile passata per argomento, aggiornandone il valore stesso.
    - **`varName`** – nome della variabile da modificare
- **`public void varSubStack(String varName)`** – Preleva l'ultimo elemento dello stack e lo sottrae dal valore della variabile passata per argomento, aggiornandone il valore stesso.
    - **`varName`** – nome della variabile da modificare
- **`public void fromStackIntoVariable(String varName)`** – Preleva l'ultimo elemento dello stack e lo salva all'interno della variabile passata per argomento
    - **`varName`** – nome della variabile da modificare
- **`public void fromVariableIntoStack(String varName)`** – Inserisce il valore della variabile passata per argomento all'interno dello stack
    - **`varName`** – nome della variabile da modificare
- **`public String toString()`** – Rappresenta la calcolatrice come stringa

Model-View-Controller



L'architettura del sistema si basa sul pattern Model-View-Controller.
Nel nostro caso il Model è composto da dalle classi "*Manager*" (*StackManager*, *VariablesManager*) che gestiscono le strutture dati e forniscono metodi di accesso e modifica.
La View mostra all'utente i dati attraverso la *UserInterface*. Nello specifico viene mostrata la text area, lo stack e le variabili, oltre alle funzioni messe a disposizione dell'utente.
Il Controller invece è realizzato attraverso la classe *Calculator*, essa riceve gli input, li elabora attraverso la classe *InputValidator* e richiede al model i dati necessari per eseguire le operazioni, oltre ad inserirne di nuovi. Infine, viene aggiornata la View presentata all'utente

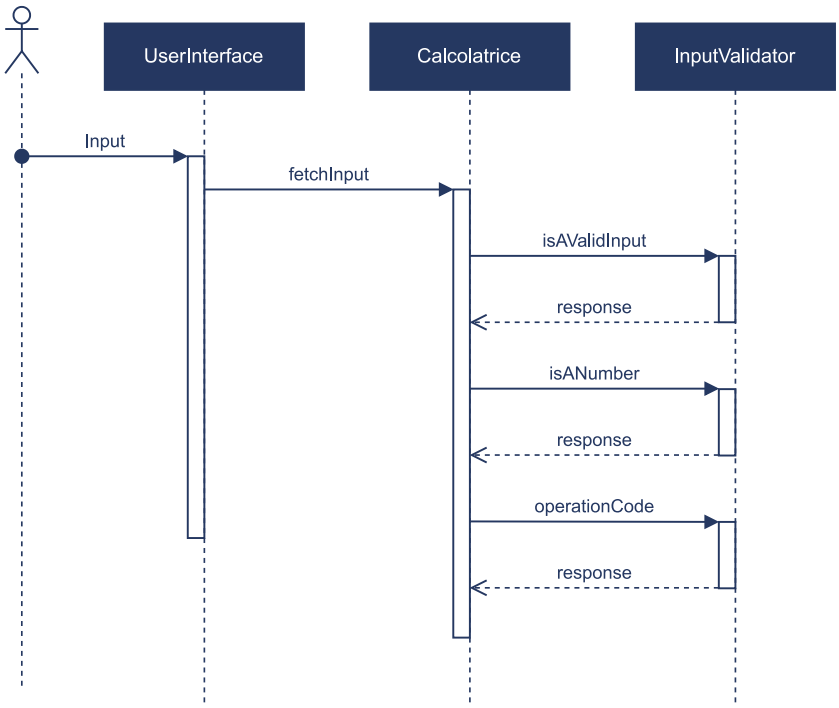
Sequence diagrams

I seguenti sequence diagram illustrano come i componenti del sistema comunichino.
Essi rispecchiano un utilizzo del sistema per mezzo della console, senza interfaccia grafica.
Il primo diagramma mostra come viene trattato l'input dell'utente. Per comprendere al meglio i successivi bisogna tenere a mente che ogni operazione (addizione, sottrazione, clear, dup, >x, <x, ...) avrà associato un codice unico, indicato dal valore di ritorno di "*OperationCode*". Quando *Calcolatrice* ottiene da *InputValidator* il codice, all'interno del metodo *fetchInput*, si passa al canalizzare il flusso del programma verso le *routines*[1] che soddisfino l'obbiettivo. L'instradamento si basa su una serie di *if-else statements* che, per l'appunto, in base al valore di *OperationCode* portano il processo verso la specifica *routine*. Quindi questi diagrammi mirano unicamente all'illustrare la comunicazione tra i componenti nelle *routines*. Si descrivono solo le *routines* più rilevanti e/o che possono essere più complesse da realizzare ed esse sono:

1. Come l'input è validato dal sistema.
2. Operazione aritmetica.
3. Memorizzazione della variabile nello stack.
4. Copia dell'ultimo elemento nello stack in una variabile.
5. Gestione delle eccezioni

Si rappresentano solo i casi di funzionamento ideali. Tuttavia, è riportato un diagramma ed una descrizione di come verranno gestite le eccezioni dal sistema.

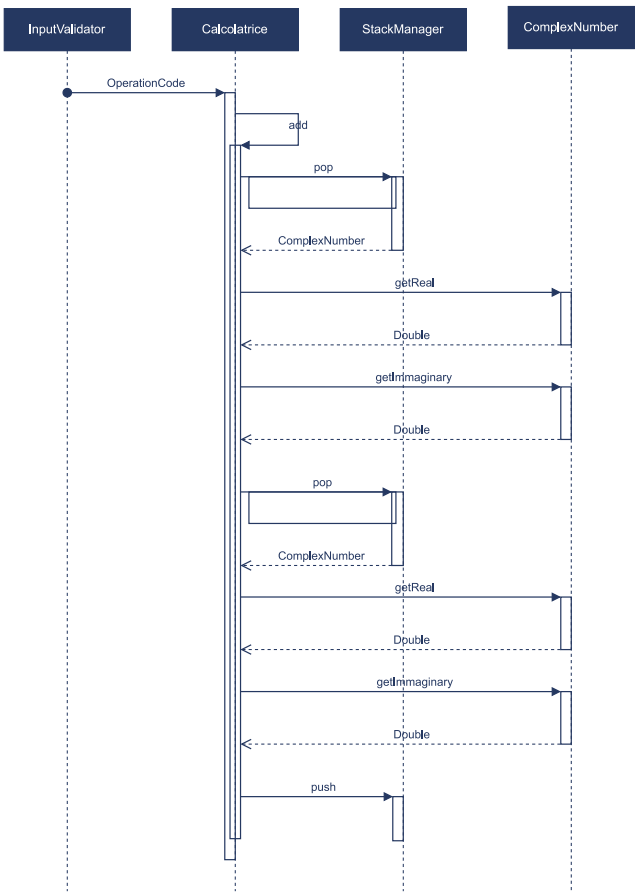
Input



[1] *Routines*: si intendono le frazioni di codice che compongono il corpo blocchi condizionali.

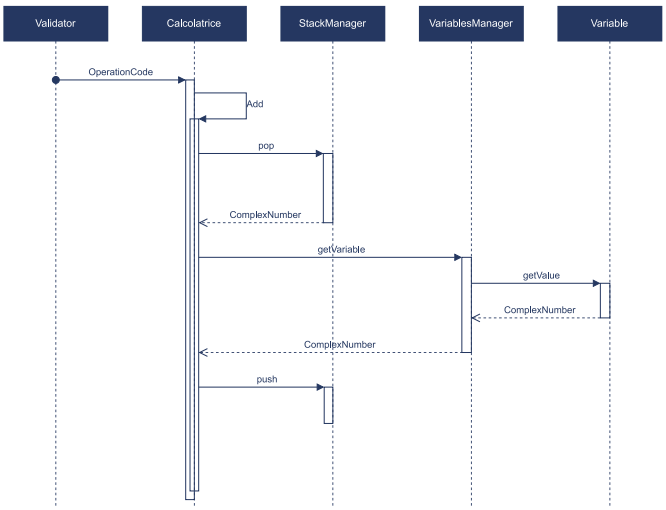
Arithmetic operations

Un'operazione aritmetica richiede al massimo due operandi che devono essere prelevati dallo stack. Qui viene illustrata un'operazione binaria, l'addizione. Questi valori sono numeri complessi, quindi, è necessario dividere la parte reale da quella immaginaria per ciascuno dei due valori. Questo passo banale verrà sottinteso nei prossimi diagrammi.



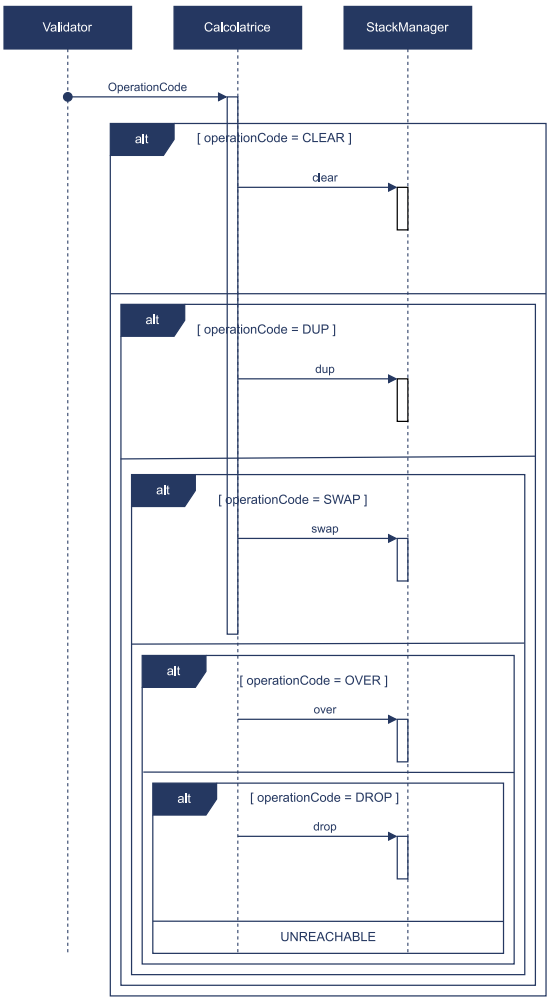
Arithmetic operations on variables

Le operazioni di somma o sottrazione sulle variabili prevedono che *Calculator* comunichi sia con *StackManager* che con *VariableManager*. Infatti, presi i due parametri tutto si riconduce all'operazione aritmetica descritta sopra.



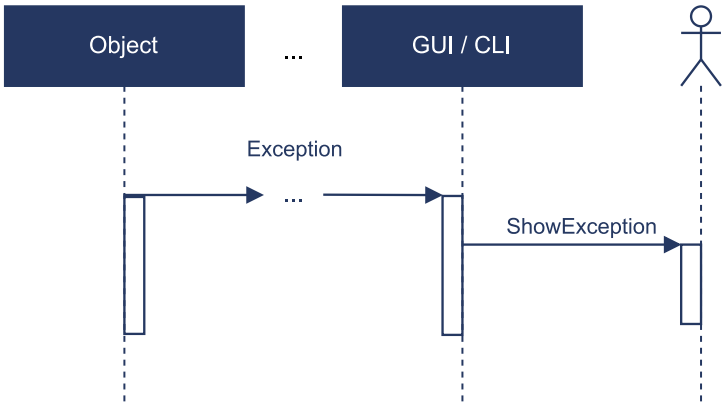
Stack operations

Le operazioni dello stack sono tutte implementate nella classe *StackManager*, quindi, la calcolatrice dovrà semplicemente invocare i metodi. Qui è esplicitato il funzionamento di *OperationCode*. Si utilizzerà la stessa metodologia nel metodo di *fetchInput* di *Calculator*.



Exceptions

Se un oggetto rileva un'eccezione, esso la propaga fino all'interfaccia utente (o la classe chiamante finché non si arriva alla funzione radice dell'albero delle chiamate a funzione).



Eccezioni

Le principali eccezioni che potrebbero essere rilevate sono le seguenti:

- StackIsFullException: Si tenta di eseguire un inserimento quando lo stack è pieno
- StackIsEmptyException: Si tenta di estrarre un elemento quando lo stack è vuoto
- EmptyVarException: Si prova a leggere un valore da una variabile ancora non inizializzata
- *ArithmeticException:* Viene eseguita una divisione il cui divisore è pari a 0
- *IllegalArgumentException:* Viene inserito un carattere non valido

Tutte le eccezioni sono di tipo *RunTimeException*, ovvero eccezioni non controllate.
Queste sono solo alcune delle possibili eccezioni, ma si mira a rilevare tutte quelle necessarie, in tal modo il programma resterà in esecuzione finché l'utente non vorrà terminarlo.

Activity Diagram

L'Activity diagram mostra il flusso del processo durante un'esecuzione del sistema e le varie fasi che lo caratterizzano.

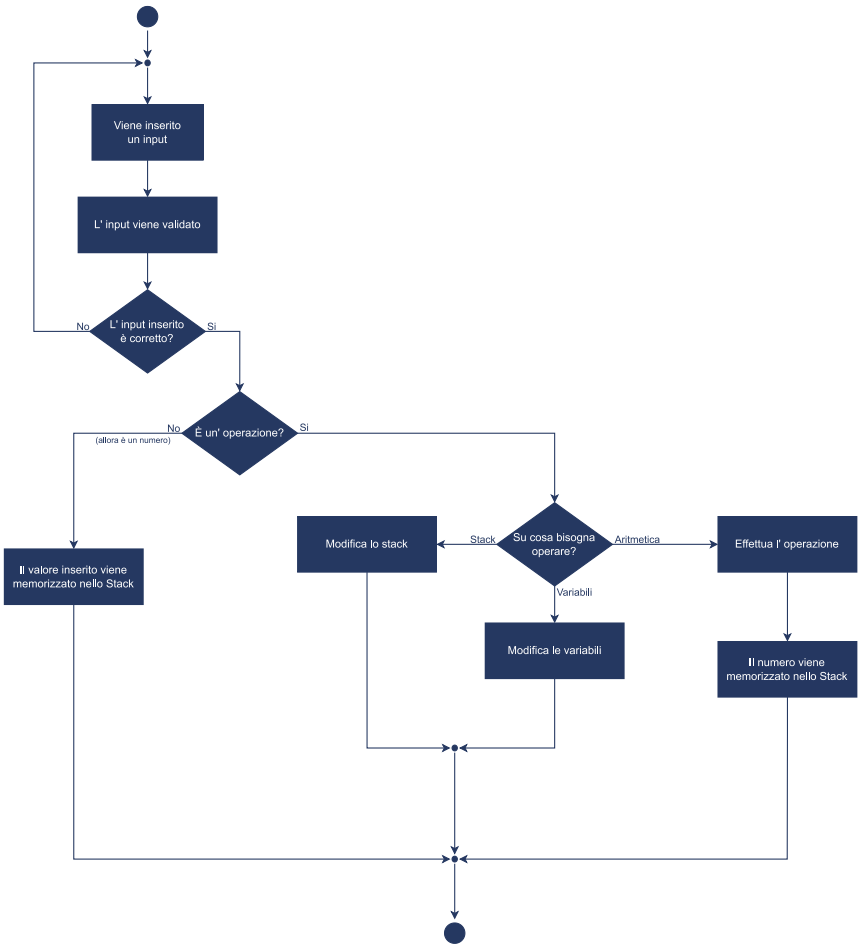


Tabella di tracciabilità

La tabella è stata aggiornata riportando il modulo del sistema dove i particolari requisiti vengono soddifatti.

#	Requirement	Design	Implementation	Testing
1.0	**Stack data structure**	StackManager		
2.0	**Input number(s) or input operation**	Calculator		
3.0	**Text Area**	UserInterface		
4.0	**Complex numbers**	ComplexNumber		
5.0	**Operazioni aritmetiche:**	Calculator		
5.1	**Addizione**	Calculator		
5.2	**Differenza**	Calculator		
5.3	**Moltiplicazione**	Calculator		
5.4	**Divisione**	Calculator		
5.5	**Radice quadrata**	Calculator		
5.6	**Inversione del segno**	Calculator		
6.0	**Flusso di esecuzione delle operazioni**	Calculator		
7.0	**Operazioni sullo stack:**	StackManager		
7.1	**CLEAR**	StackManager		
7.2	**DROP**	StackManager		
7.3	**DUP**	StackManager		
7.4	**SWAP**	StackManager		
7.5	**OVER**	StackManager		
8.0	**26 variabili**	VariablesManager		
9.0	**Operazioni sulle variabili:**	Calculator		
9.1	**Salva nella variabile (>x)**	Calculator		
9.2	**Copia nello stack (<x)**	Calculator		
9.3	**Addizione (+x)**	Calculator		
9.4	**Sottrazione (-x)**	Calculator		
10.0	**Intefaccia utente**	UserInterface		

Note conclusive

Si ribadisce che l'utilizzo del metodo `fetchInput()` all'interno della classe *Calculator* è strettamente legato all'utilizzo del sistema da console. Nel caso di sviluppo di un'interfaccia grafica, essa utilizzerà direttamente i metodi delle operazioni nella classe *Calculator*, motivo per il quale questi metodi sono tutti di visibilità pubblica.